IRIS

Indian Road Intelligence & Safe-space Planner

Complete Project Requirements & Development Specification

Core idea: Predict where danger will emerge, understand both traffic and road-surface hazards, identify the safest available drivable space, and continuously adapt the vehicle trajectory.

1. Project Vision

IRIS is a software-only autonomous-driving research prototype designed for unstructured Indian road environments. It combines multi-sensor perception, traffic and road-surface understanding, short-term motion prediction, uncertainty estimation, a future risk field, negative-space analysis, adaptive path planning, collision validation, and vehicle control in a closed loop.

2. Problem Statement

Traditional autonomous-driving systems often depend on lane markings, predictable traffic flow, standardized road geometry, and controlled intersections. Indian roads may contain mixed traffic, informal merging, missing lane markings, pedestrians crossing anywhere, wrong-side movement, cattle, pushcarts, potholes, waterlogged sections, uneven surfaces, road depressions, speed breakers, debris, and temporary obstacles. IRIS must plan safely despite this uncertainty.

3. Primary Objectives

- Detect and track diverse road users and road-surface hazards.
- Operate without depending entirely on lane markings.
- Predict short-term movement of dynamic road users.
- Represent prediction and surface uncertainty explicitly.
- Create a spatial-temporal future risk field.
- Find safe negative space / drivable regions rather than only avoiding individual objects.
- Generate collision-free and vehicle-feasible paths.
- Replan whenever the environment changes.
- Slow down or stop when no sufficiently safe trajectory exists.
- Validate performance across realistic Indian-road scenarios using measurable metrics.

4. Signature IRIS Innovations

Feature	Purpose
Future Risk Field	Estimate where danger is likely to exist in the near future, not only where obstacles are currently located.
Negative-Space Planning	Evaluate available empty/drivable regions and select the region expected to remain safest.
Traffic + Surface Intelligence	Treat vehicles, pedestrians and animals together with potholes, water, depressions, raised sections, debris
Uncertainty-Aware Planning	Increase safety margins and reduce speed when object behavior or road-surface interpretation is uncertain

Feature	Purpose
Local Road Behavior Memory	Maintain short-term environmental behavior statistics such as frequent pedestrian crossing, stopping zones

5. End-to-End Architecture

Camera + LiDAR + Radar → Sensor Fusion → Traffic Perception + Road-Surface Perception → World Model → Motion Prediction → Uncertainty Estimation → IRIS Future Risk Field → Safe-Space Analyzer → Adaptive Path Planner → Safety Validator → Vehicle Controller → Vehicle → Reassessment.

6. System Modules

Module	Requirement
A. Scenario & Environment	Create roads, intersections, market areas, village roads, highways, road boundaries, traffic actors and surface h
B. Multi-Sensor Perception	Simulate or consume camera, LiDAR and radar observations and fuse object position, velocity, geometry and c
C. Traffic Perception	Detect cars, buses, trucks, autos, motorcycles, bicycles, pedestrians, animals and pushcarts.
D. Road-Surface Perception	Detect/estimate potholes, waterlogging, depressions, raised surfaces/speed breakers, uneven road, debris and
E. Tracking	Maintain object identities and estimate velocity/heading from consecutive observations.
F. Motion Prediction	Generate short-term trajectories or probabilistic future occupancy for dynamic agents.
G. Uncertainty Estimation	Represent confidence in predicted motion and surface interpretation.
H. IRIS Risk Engine	Combine traffic, surface, collision and uncertainty risks into a spatial-temporal risk representation.
I. Safe-Space Analyzer	Identify candidate drivable regions and score their future safety.
J. Path Planning	Use global planning plus local adaptive trajectory generation.
K. Safety Validator	Check collision probability, clearance, TTC, vehicle constraints and emergency conditions.
L. Decision & Control	Select cruise, caution, replan, overtake/avoid where safe, or emergency stop; then generate steering/speed co
M. Evaluation & Dashboard	Visualize perception, risk, predictions, planned path and performance metrics.

7. Road-Surface Intelligence Requirements

- Pothole: estimate location, approximate size/severity and whether an alternate path exists.
- Waterlogged road: identify water-covered region and treat unknown depth/hidden surface as uncertainty rather than automatically assuming safe passage.
- Road depression: estimate geometric deviation where depth information is available.
- Raised surface / speed breaker: detect elevation change and adapt vehicle speed.
- Uneven road: estimate surface roughness and apply speed/path penalties.
- Debris/construction: identify as static hazards and incorporate clearance.
- Road-edge uncertainty: when boundaries are unclear, estimate drivable space from available geometry and observations.

8. Traffic & Behavior Requirements

- Handle mixed road users sharing the same space.
- Support sudden lane-free direction changes.
- Handle informal merging and stopping.
- Handle wrong-side or opposing-direction movement as a risk source.
- Handle pedestrians entering the road unexpectedly.
- Handle cattle/animals with high movement uncertainty.
- Maintain larger safety margins for high-uncertainty agents.

9. Risk Model

A conceptual cell-level risk score can combine traffic risk (T), surface risk (S), collision risk (C), and uncertainty (U): **$R(x,y,t) = w_T \cdot T + w_S \cdot S + w_C \cdot C + w_U \cdot U$**. Weights should be configurable and tuned through experiments rather than assumed to be universally optimal.

The risk engine should support temporal prediction so that a region can become high-risk before an object physically occupies it.

10. Planning Requirements

- Global planner: establish a route toward the goal.
- Local planner: generate short-horizon trajectories around dynamic hazards.
- Candidate paths must respect road boundaries, obstacle clearance and vehicle kinematics.
- Path scoring should include safety, distance, smoothness, traffic interaction, surface condition and uncertainty.
- Planner must re-evaluate when new perception arrives or risk changes.
- If no safe path exists, the safety layer must override normal path following and command controlled braking/stop.

11. Decision States

- CRUISE — environment sufficiently safe.
- CAUTION — risk increasing; reduce speed / increase margin.
- REPLAN — current trajectory no longer satisfies safety constraints.

- EMERGENCY BRAKE — no sufficiently safe trajectory or TTC is critical.
- RECOVER — resume normal planning after the hazard clears.

12. Required Scenarios

Scenario	Purpose
1. Unmarked village road	No lane markings, narrow road, mixed traffic and cattle.
2. Busy unsignalized urban intersection	Crossing vehicles, pedestrians and informal merging.
3. Highway merge	Slow-moving truck/vehicle entering or interacting with faster traffic.
4. Dense market area	High-density mixed traffic, pedestrians, pushcarts and unclear edges.
5. Sudden cattle crossing	Rapid risk increase requiring early replanning or controlled braking.
6. Pothole avoidance (recommended)	Road-surface hazard requiring lateral path adaptation.
7. Waterlogging (recommended)	Uncertain surface requiring risk-aware avoidance.
8. Water + hidden pothole (recommended)	Combined surface uncertainty and obstacle risk.
9. Wrong-side motorcycle (recommended)	Unexpected opposing movement.
10. Combined stress test (recommended)	Multiple traffic and surface hazards simultaneously.

13. Technology Stack

Area	Technology
Primary prototype	Python 3.10+
Computer vision	OpenCV, NumPy
Deep learning	PyTorch; YOLO-family detector as appropriate
Tracking/prediction	Kalman filtering initially; optional LSTM/GRU/Transformer experiments later
Planning	A* for initial global/grid planning; adaptive local trajectory planner
Simulation	Python/Pygame for initial prototype; CARLA may be used for advanced 3D validation
MathWorks target integration	MATLAB, Simulink, RoadRunner, Automated Driving Toolbox, Navigation Toolbox, Stateflow, Deep Le
Visualization	Pygame/Matplotlib initially; dashboard can be expanded later
Version control	Git + GitHub

14. Data Requirements

- Indian road imagery/video for perception testing, including the Indian Driving Dataset (IDD) where licensing/usage conditions permit.
- Synthetic data generated from the simulation for rare hazards such as water-covered potholes and cattle crossings.
- Labels/classes for traffic objects and road-surface hazards.
- Time-sequenced observations for tracking and trajectory prediction.
- Scenario metadata: actor positions, velocities, road geometry, hazard locations and ground-truth events.

15. Professional Project Structure

```
IRIS/
■■■ README.md
■■■ configs/
■■■ data/
■■■ perception/
■■■ prediction/
■■■ risk_engine/
■■■ planning/
■■■ control/
■■■ simulation/
■■■ visualization/
■■■ evaluation/
■■■ main.py
```

16. Development Phases

Phase	Scope
Phase 1 — Foundation	Road, vehicle, actors, potholes, water, basic simulation and visualization.
Phase 2 — Basic Planning	Grid representation, A*, collision checking and basic vehicle movement.
Phase 3 — IRIS Risk Engine	Traffic risk, surface risk, uncertainty and future risk field.

Phase	Scope
Phase 4 — Perception	YOLO/object detection, tracking and road-hazard detection.
Phase 5 — Prediction	Short-term trajectories and uncertainty estimation.
Phase 6 — Adaptive Planning	Safe-space selection, dynamic replanning and emergency behavior.
Phase 7 — Scenario Validation	Five mandatory Indian scenarios plus recommended stress tests.
Phase 8 — Advanced Simulation	RoadRunner/CARLA/Simulink integration where available.
Phase 9 — Evaluation	Metrics, baseline comparison, ablation experiments and failure analysis.
Phase 10 — Finalization	Dashboard, technical report, presentation and demonstration video.

17. Evaluation Metrics

- Collision count and collision rate.
- Near-collision events.
- Minimum obstacle clearance.
- Time-to-Collision (TTC) at critical events.
- Replanning latency.
- Number of replans per scenario.
- Path length and travel time.
- Path smoothness / curvature variation.
- Scenario completion rate.
- Average and minimum speed during hazards.
- Emergency braking events.
- Object detection metrics such as precision, recall and mAP where labeled data is available.
- Trajectory prediction error where ground truth is available.
- Risk-field calibration/accuracy through event-based evaluation.

18. Baseline Comparison

For a credible academic evaluation, compare IRIS against a simpler baseline such as conventional obstacle avoidance or lane/path following. Use identical scenarios and report measured differences in collisions, clearance, replanning, smoothness and completion. Do not assume IRIS is superior; the experiment should establish the result.

19. Premium Dashboard Requirements

- Live road/simulation view.
- Detected objects and classes.
- Predicted trajectories / occupancy regions.
- Future risk field heatmap.
- Safe-space candidates and selected corridor.
- Current planned path and previous path.
- Vehicle speed, steering and acceleration.
- TTC and minimum clearance.
- Current decision state: CRUISE / CAUTION / REPLAN / EMERGENCY BRAKE.
- Event log showing why replanning occurred.

20. Final Deliverables

- Complete source code with reproducible setup instructions.
- Working simulation model.
- At least five required realistic Indian-road scenarios.
- At least two detailed 3D scenes if RoadRunner is used.

- Perception, prediction, risk and planning modules.
- Evaluation dataset/scenario configuration and experiment scripts.
- Metrics and result tables/plots.
- Technical report describing architecture, algorithms, design choices, limitations and results.
- Demonstration video showing closed-loop navigation and replanning.
- Presentation with system architecture, novelty, experiments and conclusions.

21. Non-Goals / Safety Boundaries

IRIS is a simulation/research prototype, not a certified autonomous-driving system. Results in simulation must not be interpreted as evidence that a real vehicle is safe for public-road deployment. Real deployment would require extensive sensor validation, functional safety, cybersecurity, regulatory compliance, hardware-in-the-loop testing, controlled closed-course testing, and certification.

22. Master Acceptance Criteria

- The vehicle can navigate a road without relying solely on lane markings.
- Dynamic road users are represented and their short-term movement is predicted.
- Potholes, waterlogged regions and uneven/raised road sections can influence planning.
- Future risk changes when predicted occupancy or surface uncertainty changes.
- The planner chooses a safe drivable region and replans when the selected region becomes unsafe.
- The safety validator can reject unsafe trajectories.
- The vehicle slows or stops when no acceptable path exists.
- All five mandatory scenarios execute in closed loop.
- Results are logged and evaluated using objective metrics.
- The complete system can be demonstrated reproducibly from the documented project setup.

Recommended implementation order: build the simple Python foundation first, then risk/planning logic, then perception and prediction, and finally advanced 3D/MathWorks integration. This prevents the project from becoming an unmanageable collection of disconnected AI components.